SKT SOFTWARE SOLUTION	**EMPRESA DE DESARROLLO E INNOVACIÓN**	**CÓDIGO:** MI-DES-MAN-002
	PROCESO: DESARROLLO DE SOFTWARE	**VERSIÓN:** 1.0
	Desarrollo del Sistema LUPSI	**ELABORACIÓN:** 07/04/2026
	Especificación de Autenticación y Validación de Identidad	**ÚLTIMA REVISIÓN:** 07/04/2026

1. Título

Especificación Técnica de Autenticación Centralizada y Validación Algorítmica de Identidad (Módulo 10) para el Ecosistema LUPSI

2. Introducción

El sistema LUPSI prioriza una experiencia de usuario de **fricción cero** (*Zero Friction*) sin comprometer la integridad de la identidad ni la seguridad de los datos médicos. Este documento detalla la implementación técnica del motor de autenticación basado en Supabase Auth y el servicio de validación de cédula ecuatoriana mediante el algoritmo Módulo 10.

La autenticación no solo garantiza el acceso, sino que orquesta la sincronización entre la capa de seguridad gestionada de Supabase y el modelo relacional de la base de datos (perfiles y pacientes), asegurando que cada usuario autenticado tenga una identidad válida y verificable desde el primer segundo de su registro.

3. Objetivo

Documentar formalmente el funcionamiento interno, los flujos lógicos y las restricciones de seguridad aplicables al proceso de registro, inicio de sesión y validación de identidad en el backend LUPSI, sirviendo como guía de referencia técnica para mantenimiento y auditoría.

4. Tecnología y Arquitectura

- **Backend Framework**: NestJS con arquitectura de microservicios lógica.
- **Identity Provider**: Supabase Auth (Admin SDK para operaciones de registro).
- **Validation Logic**: Servicio especializado `EcuadorianIdValidatorService`.
- **Data Sync**: Sincronización atómica entre `auth.users` y `public.profiles` / `public.patients`.

5. Validación de Cédula (Algoritmo Módulo 10)

El servicio `EcuadorianIdValidatorService` implementa una validación matemática de tres capas para garantizar que el DNI ingresado sea un documento verídico emitido por el Registro Civil del Ecuador.

5.1. Reglas de Validación

1. **Longitud y Formato**: Debe contener exactamente 10 dígitos numéricos.
2. **Código de Provincia**: Los dos primeros dígitos deben estar en el rango [01, 24] o ser el código 30 (Ecuatorianos en el exterior).
3. **Tipo de Persona**: El tercer dígito debe ser menor a 6, indicando que el DNI pertenece a una persona natural y no a una entidad jurídica (RUC).
4. **Algoritmo de Módulo 10**: Validación del dígito verificador.

5.2. Lógica del Algoritmo

El proceso matemático sigue estos pasos:

- Se extraen los primeros 9 dígitos.
- Se multiplican por los coeficientes fijos: `[2, 1, 2, 1, 2, 1, 2, 1, 2]`.
- Si el resultado de una multiplicación es ≥ 10, se le resta 9.
- Se suman todos los resultados parciales (`totalSum`).
- Se calcula la decena superior inmediata de `totalSum`.
- El dígito verificador resultante debe ser igual a la diferencia entre la decena superior y la suma total.

Listing 1: Extracto de implementación del validador

```
const coefficients = [2, 1, 2, 1, 2, 1, 2, 1, 2];
for (let i = 0; i < 9; i++) {
let product = parseInt(dni.charAt(i)) * coefficients[i];
if (product >= 10) product -= 9;
```

```
totalSum += product;
}
const nextTen = Math.ceil(totalSum / 10) * 10;
const expectedDigit = (nextTen - totalSum) === 10 ? 0 : (nextTen - total
```

6. Flujo de Registro "Fricción Cero"

El `AuthService.register` orquesta una transacción distribuida entre el servicio de identidad de Supabase y las tablas relacionales de la base de datos.

6.1. Secuencia de Operaciones

Paso	Acción	Descripción
1	**Validación ID**	Ejecución del Módulo 10. Si falla, el flujo termina con Error 400.
2	**Creación en Auth**	Se crea el usuario en `supabase.auth` con auto-confirmación de email activada.
3	**Sincronización Profile**	Inserción en `public.profiles` con el UUID generado por Auth.
4	**Sincronización Patient**	Inserción de DNI y metadatos en `public.patients`.
5	**Auto-Login**	Se genera la sesión JWT inmediata para que el usuario sea redirigido al Dashboard sin re-autenticarse.

6.2. Estrategia de Rollback y Robustez

Para mantener la integridad referencial, si la inserción en las tablas de base de datos (`profiles` o `patients`) falla (por ejemplo, duplicidad de cédula detectada por constraint UNIQUE), el backend solicita inmediatamente el **borrado del usuario de Auth** mediante el Admin SDK de Supabase, evitando "usuarios huérfanos" en el sistema de seguridad que no existan en el sistema relacional.

7. Módulo de Sesiones (Login)

El inicio de sesión delega la autenticación criptográfica a Supabase, pero el backend actúa como guardián de la lógica de negocio.

- **Carga de Claims**: El JWT resultante contiene el identificador único del usuario (`sub`) y se orquesta con las políticas RLS para acceso a datos.
- **Manejo de Errores**: Los intentos fallidos son capturados y estandarizados para no revelar información sobre la existencia de correos electrónicos en la base de datos (Seguridad por Ofuscación).

8. Responsabilidades

Responsable	Rol en el Módulo
Daniel Luisa	Implementación de lógica NestJS, Integración Supabase SDK y validadores.
Sebastián Ortiz	Diseño de transaccionalidad, constraints SQL y revisión de seguridad.

9. Firmas de Responsabilidad

Rol	Nombre / Representante	Firma (QR)	Fecha
Gestor del Proyecto	Angel Ayuquina		07/04/2026
Arquitecto BD y Cloud	Sebastián Ortiz		07/04/2026
Desarrollador Backend y QA	Daniel Luisa		07/04/2026
Desarrollador Frontend y UX/UI	Alex Guachi		07/04/2026